# Architettura degli Elaboratori a.a. 2025/26

## Appello d'esame 14/09/2026 (ASM)

---

### Operazioni preliminari

1. Aprire tramite RARS il file `program01.asm` in questa directory
2. Completare le seguenti righe con i propri dati in `program01.asm`

```
########################################
# INSERIRE I PROPRI DATI QUI:
# Nome:
# Cognome:
# Matricola:
########################################
```

### Esercizio

Si realizzi in assembly RISC-V il seguente programma. Sia data in input da utente una stringa di caratteri `S`. La stringa è lunga al più 24 caratteri (non è necessario verificare questo vincolo) e consiste dei seguenti simboli: '`0`' (codice ASCII `0x30`), '`1`' (codice ASCII `0x31`), '`-`' (codice ASCII `0x2D`). La sequenza termina quando viene letto il carattere `'\n'` (codice ASCII `0xA`). Il programma deve considerare ogni sequenza di caratteri separata da '`-`' come un numero binario ed effettuare l'operazione corrispondente di sottrazione ('`-`') come rappresentato nella stringa di input e stampare a schermo la rappresentazione decimale del risultato. Il carattere '`-`' non può comparire come primo o ultimo carattere della sequenza (quindi si suggerisce di prestare attenzione al primo numero: mentre tutti i numeri successivi saranno sottratti, il primo numero sarà sempre positivo).

Se, ad esempio, `S` è `11-0000-1-0010` allora dovrà essere scritto a schermo:

```
0
```

perché la sequenza corrisponde in decimale all'espressione 3-0-1-2=0. Notare che ogni numero binario può contenere zeri iniziali ridondanti (è la presenza del separatore '`-`' a indicare la lunghezza di ogni numero binario).

Nella sezione dedicata al **text segment**, il programma deve avere nel comparto *main* il caricamento dei dati da input (già fornito nel file `program01.asm`: l'indirizzo base della stringa `S` verrà caricato nel registro `a0`), e il codice necessario a calcolare i valori e visualizzare i risultati sullo schermo.

Note: Commentare le righe più importanti del codice avendo cura di spiegare a cosa servano i registri. Si ricorda che la syscall per la stampa di un intero in notazione decimale prevede come argomenti `a7=1` e `a0` assegnato con il valore da stampare.

SUGGERIMENTO: Si ricorda che il comando per lo shift logico a sinistra di tipo immediate è `slli`.

### Risultato atteso

Per ogni input al programma va stampato l'output della procedura suddetta. Ad esempio, il file `test-03.in` contiene il seguente input: '`1-1001-11-0011-000`', il suo output atteso (all'interno di `test-03.expt`) è:

```
-14
```

È possibile studiare i casi di test aprendo i file di input (nel formato `test-`*xy*`.in`, dove *xy* è un numero a due cifre) e output atteso (estensione `test-`*xy*`.expt`), confrontandoli col proprio (estensione `test-`*xy*`.out`).

### Verifica di corretta esecuzione dell'esercizio e Consegna

Per verificare che l'esercizio sia stato completato correttamente eseguire `run.sh` (doppio click sul file dal file manager, oppure esecuzione del comando `./run.sh` da terminale) e visualizzare il risultato aprendo il file `test_results.html`. Questo comando copierà l'elaborato nella cartella di rete situata nella scrivania `esame/esameASM/consegna`. Verificare che la cartella contenga correttamente il file da consegnare.

### Bonus

Una soluzione pienamente funzionante, dove ogni test verrà completato usando meno di 220 istruzioni sarà premiata con un bonus di **1 punto** sul voto finale.

### Note

- Non è consentito modificare il *data segment*.
- Il limite massimo di istruzioni eseguibili è `305900`. Oltre quel numero, l'esecuzione viene automaticamente terminata.
- Il file `program01.vuoto.asm` contiene una copia del file `program01.asm` che può essere utile in caso sia necessario ripartire da capo.
- Attenzione a non eseguire loop infiniti con leak della memoria: RARS potrebbe crashare e cancellare il file che state scrivendo, sentitevi liberi di salvare un file di backup prima di eseguire del codice rischioso.